

Longest Common Subsequence (LCS)

Algorithm Design & Analysis | CSCI208 — Spring 2026

Brute Force

Recursive

Dynamic Programming

1. Introduction

The Longest Common Subsequence (LCS) is a classic computer science problem that finds the longest sequence of characters appearing in the same relative order in two strings, without requiring the characters to be consecutive. This project implements and compares three algorithms for solving the LCS problem: Brute Force, Recursive (with memoization), and Dynamic Programming.

2. Problem Definition

Given two strings **s1** of length **n** and **s2** of length **m**, find the longest string **L** such that every character of **L** appears in **s1** in order and also appears in **s2** in order, with maximum possible length.

Example

String 1	String 2	LCS	Length
ABCB DAB	BDCABA	BDAB	4

Key properties of a subsequence:

- Characters must remain in their original order (no rearranging).
 - Characters do NOT need to be consecutive.
 - There may be more than one valid LCS of the same length.
-

3. Algorithms Implemented

3.1 Brute Force

The brute force approach generates every possible subsequence of **s1** (there are 2^n of them), then checks whether each one also appears as a subsequence of **s2**. The longest match is returned.

Steps:

- Generate all 2^n subsequences of **s1**.
- For each subsequence, scan **s2** to check if it appears in order.
- Track and return the longest one found.

```
def get_subsequences(s):
    result = [""]
    for ch in s:
        result += [x + ch for x in result]
    return result
def brute_force_lcs(s1, s2):
    best = ""
    for sub in get_subsequences(s1):
        if is_subsequence(sub, s2) and len(sub) > len(best):
            best = sub
```

return best	
Metric	Value
Time Complexity	$O(2^n \times m)$
Space Complexity	$O(2^n)$
Practical Limit	~18 characters

3.2 Recursive (with Memoization)

The recursive approach compares characters from the end of both strings. If they match, the character is included and both pointers move inward. If not, two branches are explored and the longer result is kept. Memoization stores already-computed results to avoid exponential re-computation.

Recurrence relation:

```
def recursive_lcs(s1, s2, m, n, memo):
    if m == 0 or n == 0: return ""
    if (m, n) in memo: return memo[(m, n)]
    if s1[m-1] == s2[n-1]:
        result = recursive_lcs(s1, s2, m-1, n-1, memo) + s1[m-1]
    else:
        a = recursive_lcs(s1, s2, m-1, n, memo)
        b = recursive_lcs(s1, s2, m, n-1, memo)
        result = a if len(a) > len(b) else b
    memo[(m, n)] = result
    return result
```

Metric	Value
Time Complexity (no memo)	$O(2^{(n+m)})$
Time Complexity (with memo)	$O(n \times m)$
Space Complexity	$O(n \times m)$ memo + $O(n+m)$ stack

Important note: The memo dictionary is passed as a plain argument (not a default argument) to avoid the mutable default argument bug in Python, where a shared default dictionary would persist across separate function calls and return wrong results.

3.3 Dynamic Programming

Dynamic Programming builds a 2D table `dp` where `dp[i][j]` stores the LCS length for the prefixes `s1[:i]` and `s2[:j]`. Each cell is computed exactly once using the results of previously filled cells. The actual LCS string is reconstructed by tracing back through the table.

Fill rule:

```
if s1[i-1] == s2[j-1]:
    dp[i][j] = dp[i-1][j-1] + 1
else:
```

```
dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

Example table for s1="ABCB" and s2="BCB":

	e	B	C	B
e	0	0	0	0
A	0	0	0	0
B	0	1	1	1
C	0	1	2	2
B	0	1	2	3

Trace back from $dp[4][3]=3$ gives LCS = "BCB".

Metric	Value
Time Complexity	$O(n \times m)$
Space Complexity	$O(n \times m)$
Practical Limit	Any length

4. Performance Results

The following table shows measured execution times in milliseconds across increasing input lengths, using random strings with a small alphabet (a–e) to ensure meaningful matches.

Length (n)	Brute Force (ms)	Recursive (ms)	DP (ms)
5	0.03	0.01	0.02
8	0.24	0.18	0.02
10	0.72	1.54	0.03
12	3.02	4.64	0.04
15	32.26	633.40	0.05
18	314.67	7771.0	0.09
20	skipped	skipped	0.10

Dynamic Programming remains under 0.1 ms for all tested sizes. Both Brute Force and Recursive grow exponentially — Recursive reaches over 7 seconds at $n=18$, while DP completes the same input in under 0.1 ms.

5. Complexity Summary

Algorithm	Time Complexity	Space Complexity	Practical Limit
Brute Force	$O(2^n \times m)$	$O(2^n)$	~18 chars
Recursive (memo)	$O(n \times m)$	$O(n \times m)$	~100+ chars
Dynamic Programming	$O(n \times m)$	$O(n \times m)$	Any length

6. Team Contributions

Member	Role	Responsibilities
Member 1	Algorithm Developer	Implemented all three LCS functions in Python
Member 2	Performance Tester	Built benchmark script, collected timing data
Member 3	Report Author	Wrote this report and complexity analysis
Member 4	Presentation Designer	Created slide deck and visual examples
Member 5	Tester & Demo Lead	Tested edge cases, prepared discussion demo

7. Conclusion

This project demonstrated three fundamentally different approaches to solving the LCS problem. Brute Force is conceptually simple but impractical beyond very short inputs. The Recursive approach correctly models the problem but suffers from exponential overlap without memoization. Dynamic Programming eliminates all redundant computation by storing results in a table, achieving polynomial time $O(n \times m)$ and making it the clear choice for any real-world use.

Real-world applications of LCS:

- Version control systems (e.g. git diff)
 - DNA and RNA sequence alignment in bioinformatics
 - Spell checkers and autocomplete engines
 - File comparison and merge tools
-

8. References

- Cormen, T. H., et al. Introduction to Algorithms (CLRS), 4th Edition — Chapter 15
- CSCI208 Lecture Notes, Spring 2026
- Python Documentation — docs.python.org